9dfee015-98cd-4643-b93b-0a94a9da5677

<prev_user>

<request>

You are working on an issue in the 'jaiarobotics/jaiabot' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/jaiabot/jaiabot (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-03-24T01:22:52.530Z</current_datetime>

Consider the following problem statement:

<problem_statement>
Please try again
</problem_statement>

Implement the necessary changes to the repository so that the requirements specified in the <problem_statement> are met.

<prev_assistant>

Copilot had to stop work due to a timeout.

<current>

<request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'jaiarobotics/jaiabot' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/jaiabot/jaiabot (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-03-24T02:51:15.840Z</current_datetime>

Consider the following problem statement:

<problem_statement>

This section includes details on the pull request with code changes you have already submitted to fix the problem above.

<pr_title>
Add jaia_state_tool: compile-time Clang 18 AST analysis of boost::statechart state machines
</pr_title>

<pr_description>
- [ ] Add `ChoiceTransition` struct and `choices` field to `StateInfo`
- [ ] Add AST visitor helpers: `findTransitTarget`, `extractCaseCondition`, `extractChoicesFromDecl` to walk `react()` switch statements
- [ ] Call `extractChoicesFromDecl` from `processState` for choice pseudostates
- [ ] Update `generateYAML` to emit `choices:` block
- [ ] Update `generateDOT` to render choice states as diamonds and emit choice transitions
- [ ] Update `generateMermaid` to emit `<<choice>>` declarations and choice transitions with condition labels
</pr_description>

Comments on the PR (you are @copilot in this section)

<comments>

<pr_comments>

```
e_ReacquireGPS
            state "ResolveNoForwardProgress" as
jaiabot_statechart_inmission_pause_ResolveNoForwardProgress
        }
        state "Underway" as
jaiabot_statechart_inmission_Underway {
            state "Abort" as
jaiabot_statechart_inmission_underway_Abort
            state "Movement" as
jaiabot_statechart_inmission_underway_Movement {
                [*]  jaiabot_statechart_PreDeployment
    state "InMission" as jaiabot_statechart_InMission {
        state "{this}" as jaiabot_statechart_InMission_this
        state "Pause" as jaiabot_statechart_inmission_Pause {
            state "IMURestart" as
jaiabot_statechart_inmission_pause_IMURestart
            state "Manual" as
jaiabot_statechart_inmission_pause_Manual
            state "ReacquireGPS" as
jaiabot_statechart_inmission_pause_ReacquireGPS
            state "ResolveNoForwardProgress" as
jaiabot_statechart_inmission_pause_ResolveNoForwardProgress
        }
        state "Underway" as
jaiabot_statechart_inmission_Underway {
            state "Abort" as
```

```
jaiabot_statechart_inmission_underway_Abort
            state "Movement" as
jaiabot_statechart_inmission_underway_Movement {
                [*] --&gt;
jaiabot_statechart_inmission_underway_movement_MovementSelection
   ...
</comment_old>

<comment_old>
<author>@copilot</author>

&gt;
&gt;
&gt; Make updates to the AST generator:...

Done in commit `3ec4737`. Here's what was implemented:

**AST/YAML changes:**
1. **`type: choice` for `*Selection` states** — any state whose
unqualified name ends with `"Selection"` (e.g.
`MovementSelection`, `TaskSelection`) gets `type: choice` in the
YAML. DOT renders them as filled black diamonds; Mermaid uses
`state id <<choice>>`.
2. **`initial_state` for every composite state** — fixed a
filter bug where `child.find("statechart::")` was incorrectly
rejecting initial children in the `jaiabot::statechart::*`
namespace. Changed to `child.find("boost::statechart::")` so
only boost internal pseudo-types (like
`boost::statechart::has_deep_history`) are filtered. This makes
`[*] --> MovementSelection` and similar entries appear
correctly. The YAML key is also consistently named
`initial_state` for all composite states (was `initial_child`).

**Mermaid-only changes:**
1. **`{this}` pseudo-state** — each composite state block now
includes `state "{this}" as id_this`. All outgoing transitions
from that composite state are drawn `FROM id_this` rather than
from the composite state border, so lines originate from inside
the box:
   ```
   state "InMission" as jaiabot_statechart_InMission {
       state "{this}" as jaiabot_statechart_InMission_this
       ...
   }
   jaiabot_statechart_InMission_this Make updates to the AST
generator:...
```
```

Done in commit *****__CODE_BLOCK_1_0__*****. Here's what was implemented:

AST/YAML changes:
1. *******__CODE_BLOCK_1_1__***** for *****__CODE_BLOCK_1_2__***** states** — any state whose unqualified name ends with *****__CODE_BLOCK_1_3__***** (e.g. *****__CODE_BLOCK_1_4__*****, *****__CODE_BLOCK_1_5__*****) gets *****__CODE_BLOCK_1_6__***** in the YAML. DOT renders them as filled black diamonds; Mermaid uses *****__CODE_BLOCK_1_7__*****.
2. *******__CODE_BLOCK_1_8__***** for every composite state** — fixed a filter bug where *****__CODE_BLOCK_1_9__***** was incorrectly rejecting initial children in the *****__CODE_BLOCK_1_10__***** namespace. Changed to *****__CODE_BLOCK_1_11__***** so only boost internal pseudo-types (like *****__CODE_BLOCK_1_12__*****) are filtered. This makes *****__CODE_BLOCK_1_13__***** and similar entries appear correctly. The YAML key is also consistently named *****__CODE_BLOCK_1_14__***** for all composite states (was *****__CODE_BLOCK_1_15__*****).

Mermaid-only changes:
1. *******__CODE_BLOCK_1_16__***** pseudo-state** — each composite state block now includes *****__CODE_BLOCK_1_17__*****. All outgoing transitions from that composite state are drawn *****__CODE_BLOCK_1_18__***** rather than from the composite state border, so lines originate from inside the box:
   *****__CODE_BLOCK_1_19__*****
   state "InMission" as jaiabot_statechart_InMission {
       state "{this}" as jaiabot_statechart_InMission_this
       ...
   }
   jaiabot_statechart_InMission_this -->
jaiabot_statechart_inmission_underway_Abort : EvAbort
   ```
2. **`[*] -->` entry arrows** — now appear for every composite state's initial child once the filter bug is fixed (e.g. `[*] -->
jaiabot_statechart_inmission_underway_movement_MovementSelection` inside `Movement`).
</comment_old>

<comment_old>
<author>@tsaubergine</author>
1. The choice states are not implemented correctly - they don't
   ```

````
show up in the YAML and are not rendered correctly. See below
for what I would like to see for those in the .mmd output.

2. Also, please use {this} only for:

-  head when going from parent to child state
- tail when going from child to parent state

3. Use "direction LR" in Mermaid

This is what I want the mermaid output to look like:
```
stateDiagram-v2
    direction LR
    [*]
1. The choice states are not implemented correctly - they don't
show up in the YAML and are not rendered correctly. See below
for what I would like to see for those in the .mmd output.

2. Also, please use {this} only for:

-  head when going from parent to child state
- tail when going from child to parent state

3. Use "direction LR" in Mermaid

This is what I want the mermaid output to look like:
*****__CODE_BLOCK_1_0__*****
stateDiagram-v2
    direction LR
    [*] --> jaiabot_statechart_PreDeployment
    state "InMission" as jaiabot_statechart_InMission {
        state "{this}" as jaiabot_statechart_InMission_this
        [*]  jaiabot_statechart_PreDeployment
    state "InMission" as jaiabot_statechart_InMission {
        state "{this}" as jaiabot_statechart_InMission_this
        [*] --> jaiabot_statechart_inmission_Underway
        state "Pause" as jaiabot_statechart_inmission_Pause {
            state "{this}" as
jaiabot_statechart_inmission_Pause_this
            [*]  jaiabot_statechart_inmission_Underway
        state "Pause" as jaiabot_statechart_inmission_Pause {
            state "{this}" as
jaiabot_statechart_inmission_Pause_this
            [*] --> jaiabot_statechart_inmission_pause_Manual
            state "IMURestart" as
jaiabot_statechart_inmission_pause_IMURestart
````
````

```
            state "Manual" as
jaiabot_statechart_inmission_pause_Manual
            state "ReacquireGPS" as
jaiabot_statechart_inmission_pause_ReacquireGPS
            state "ResolveNoForwardProgress" as
jaiabot_statechart_inmission_pause_ResolveNoForwardProgress
        }
        state "Underway" as
jaiabot_statechart_inmission_Underway {
            state "{this}" as
jaiabot_statechart_inmission_Underway_this
            [*]  jaiabot_statechart_inmission_pause_Manual
            state "IMURestart" as
jaiabot_statechart_inmission_pause_IMURestart
            state "Manual" as
jaiabot_statechart_inmission_pause_Manual
            state "ReacquireGPS" as
jaiabot_statechart_inmission_pause_ReacquireGPS
            state "ResolveNoForwardProgress" as
jaiabot_statechart_inmission_pause_ResolveNoForwardProgress
        }
        state "Underway" as
jaiabot_statechart_inmission_Underway {
            state "{this}" as
jaiabot_statechart_inmission_Underway_this
            [*] --&gt;
jaiabot_statechart_inmission_underway_Movement
            state "Abort" as
jaiabot_statechart_inmission_underway_Abort
            state "Movement" as
jaiabot_statechart_inmission_underway_Movement {
                state "{this}" as
jaiabot_statechart_inmission_underway_Movement_this
                [*]
jaiabot_statechart_inmission_underway_Movement
            state "Abort" as
jaiabot_statechart_inmission_underway_Abort
            state "Movement" as
jaiabot_statechart_inmission_underway_Movement {
                state "{this}" as
jaiabot_statechart_inmission_underway_Movement_this
                [*] --&gt;
jaiabot_statechart_inmission_underway_movement_MovementSelection
                state "MovementSelection" as
jaiabot_statechart_inmission_underway_movement_MovementSelection

                state "RemoteControl" as
```

```
jaiabot_statechart_inmission_underway_movement_RemoteControl {
                    state "{this}" as jai...
</comment_old>

<comment_old>
<author>@tsaubergine</author>
@copilot+claude-sonnet-4.6 please try again
</comment_old>

<comment_old>
<author>@copilot</author>

&gt; @copilot+claude-sonnet-4.6 please try again

Done in commit `dcc053b`:

1. **Choice states**: Changed from anonymous `state id
<<choice>>` to labeled `state "Name" as id <<choice>>` so the
state name is visible in the rendered diagram.

2. **`{this}` restricted to parent/child**: Added
`isDescendantMmd()` helper and changed the rule:
   - Source uses `{this}` only when the source is composite
**and** the target is a descendant (parent → child)
   - Destination uses `{this}` only when the target is composite
**and** the source is a descendant (child → parent)
   - Sibling-to-sibling transitions (e.g. `Movement → Task`) now
use plain IDs

3. **`direction LR`**: Added after `stateDiagram-v2`.
</comment_old>

<comment_old>
<author>@tsaubergine</author>
I need you to update the AST parser so that the TaskSelection
and other *Selection states actually show up in the YAML output
(they do not now) and then ultimately the "choice" output in the
generated graphs.
</comment_old>

<comment_old>
<author>@copilot</author>

&gt; I need you to update the AST parser so that the
TaskSelection and other *Selection states actually show up in
the YAML output (they do not now) and then ultimately the
"choice" output in the generated...
```

Fixed in commit `f9aed32`. Three changes to ensure `*Selection` states appear in the YAML and diagrams:

1. **`resolveChildStates()` post-processing** — after AST traversal completes, this new function checks every composite state's `initial_child` field. If the referenced child (e.g. `TaskSelection`, `MovementSelection`) is not yet in `g_states`, it adds a stub entry with the correct `parent` and `is_choice = true` (detected by the `*Selection` naming convention). This is the primary fix for the case where the AST visitor misses the state definition.

2. **Single non-list reaction extraction** — `extractReactionsFromDecl` now handles `using reactions = boost::statechart::custom_reaction<E>` (not wrapped in `mpl::list`) in addition to `boost::mpl::list<...>`. Previously only `mpl::list` reactions were extracted, so `TaskSelection`/`MovementSelection` showed no reactions.

3. **Stub overwrite** — `processState` now overwrites a stub entry (one with empty reactions) when the real class definition is visited, ensuring the full info takes priority if the primary path does succeed.
</comment_old>

<comment_old>
<author>@tsaubergine</author>
I'm still having trouble with the "choice" states (*Selection states).

I've updated the branch to remove the code that I don't think is useful. Let's try a new approach to getting these choice states to correctly render.

For "choice" states, I want you to define an AST visitor that finds the "react" function and walks through the switch statement extracting the case and transit class to use for the various choice outputs.

For example, given MovementSelection:

```
struct MovementSelection :
boost::statechart::state<MovementSelection, Movement>,
                         AppMethodsAccess<MovementSelection>
{
```

```
    struct EvMovementSelect :
boost::statechart::event<EvMovementSelect>
    {
    };

    using StateBase =
boost::statechart::state<MovementSelection, Movement>;
    MovementSelection(typename StateBase::my_context c) :
StateBase(c)
    {
        post_event(EvMovementSelect());
    }
    ~MovementSelection() {}

    boost::statechart::result react(const EvMovementSelect&)
    {
        switch (this->machine().mission_plan().movement())
        {
            case protobuf::MissionPlan::TRANSIT: return
transit<Transit>();
            case protobuf::MissionPlan::REMOTE_CONTROL: return
transit<RemoteControl>();
            case protobuf::MissionPlan::TRAIL: return
transit<Trail>();
        }

        // should never reach here but if does, abort the
mission
        return transit<underway::Abort>();
    }

    using reactions =
boost::statechart::custom_reaction<EvMovementSelect>;
};
```

Find the "react" function, then search its switch statement to conceptually extract pairs like:

jaiabot::protobuf::MissionPlan::TRANSIT → Transit
jaiabot::protobuf::MissionPlan::REMOTE_CONTROL → RemoteControl
jaiabot::protobuf::MissionPlan::TRAIL → Trail

Given that, then in the YAML output I want to see something like this with a new "choices" tab showing these conditional transitions:

```
  - name:
jaiabot::statechart::inmission::underway::movement::MovementSele
ctio...
</comment_old>

<comment_new>
<comment_id>4115013109</comment_id>
<author>@tsaubergine</author>
&gt; I'm still having trouble with the "choice" states
(*Selection states).
&gt;
&gt; I've updated the branch to remove the code that I don't
think is useful. Let's try a new approach to getting these
choice states to correctly render.
&gt;
&gt; For "choice" states, I want you to define an AST
visitor that finds the "react" function and walks through
the switch statement extracting the case and transit class
to use for the various choice outputs.
&gt;
&gt; For example, given MovementSelection:
&gt;
&gt; ```
&gt; struct MovementSelection : boost::statechart::state</
comment_new>

</pr_comments>

</comments>

----
The last **13** git commits in this branch are the changes
you have made so far. Use those as your change commit
history.
</problem_statement><rules_for_making_changes>
* Start by fully understanding <problem_statement> and the
<comments> before making any changes. You will only address
the <comment_new> comments. Reply to new comments as you
address them. Ignore comments that are not actionable or
directed at you.
* For each new comment, determine:
    - If it a request for changes, a question on the code or
a suggestion.
    - Is it addressed to you or someone else, e.g. "Hey
@username, ..." or "@username can you ...".
    - Is it a general comment or praise.
* If there are no comments you need to address or reply to,
```

```
you can stop now without exploring or explaining yourself.
* If the user's ask is not clear, you can reply to the
comment and ask for clarification.
* If an action or explanation is needed, follow the steps
below.
</rules_for_making_changes>
```